

CS599 Biometrics

Lab 01 on Python, NumPy and Pandas

01/22/2026

Objective of this lab is to explore the NumPy and Matplotlib packages that provide capabilities for handling vectors, matrices, and Matlab-like plotting.

In []:

Based on Anaconda.com tutorials

References

- [Python Tutorial](#) - a good next step to learning Python (about a 2-3 hour commitment)
- [Python Reserved Words](#) - about 40 terms that form the base grammar of the language.
NOTE: These are not functions.
- [Python builtin module](#) functions (60) and classes (20)
- [Python Standard Library](#)
- [Python Package Index](#) community contributed Python packages
- [Idiomatic Python](#)

Jupyter Hotkeys

There are many hotkeys to facilitate typing in Jupyter, here are a few of them:

Jupyter Hotkey	Result
Enter	edit cell
Ctrl+Enter	run cell
Shift+Enter	run cell , go to next cell
a	add box above
b	add box below

Jupyter Hotkey	Result
y	to code
m	to markdown
r	to raw
c	copy cell
x	cut cell
d d	delete cell (click d twice)
v	paste cell below
Shift+v	paste cell above
o	toggle output
i i	interrupt kernel (click i twice)

Note, a cell is selected with the leftbar is blue, and is being edited when it is green

Downloading and installing python, TensorFlow and Anaconda

<https://www.python.org/>

<https://www.tensorflow.org/>

<https://www.anaconda.com/>

Numpy

NumPy is the fundamental package for scientific computing in Python. It is a Python library that provides a multidimensional array object, various derived objects (such as masked arrays and matrices), and an assortment of routines for fast operations on arrays, including mathematical, logical, shape manipulation, sorting, selecting, I/O, discrete Fourier transforms, basic linear algebra, basic statistical operations, random simulation and much more.

- a powerful N-dimensional array object that facilitate operations on large numbers of data.
- sophisticated (broadcasting) functions
- tools for integrating C/C++ and Fortran code
- useful linear algebra, Fourier transform, and random number capabilities
- Many Python-based scientific and mathematical packages use NumPy array functionality

```
In [1]: a = 2
```

In [2]: `!pip install numpy`

Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: numpy in c:\users\mleen\appdata\roaming\python\python313\site-packages (2.3.2)

[notice] A new release of pip is available: 25.2 -> 25.3

[notice] To update, run: python.exe -m pip install --upgrade pip

In [3]: `import numpy as np`
*# import numpy * -> imports all numpy function and can be called without prefix, i.*

`s1 = np.array([1, 4, 9, 16, 25])` *#create a row of five elements*

In [4]: `s2 = np.array([[1,2,3,4,5],
 [1,4,9,16,25],
 [1,8,27,64,125]])` *# array of three rows and five columns*

In [5]: `s3 = np.array([2,5,10,17,26])`

In [6]: `s1`

Out[6]: `array([ 1, 4, 9, 16, 25])`

In [7]: `s2`

Out[7]: `array([[1, 2, 3, 4, 5],
 [1, 4, 9, 16, 25],
 [1, 8, 27, 64, 125]])`

Square brackets allow us to extract an element of the array at the given index. Note, the first element of the array is defined as 0, NOT 1

In [8]: `s1[3]`

Out[8]: `np.int64(16)`

You can extract multiple elements at once. Note, the final index is NOT included

In [9]: `v1 = s1[1:4]` *# different from Python list slice which makes a copy.*
Numpy just copies references
: means indexes from 1 to 4 but not including 4

In [10]: `v1`

Out[10]: `array([ 4, 9, 16])`

In [11]: `s1[3] = 99` *# you can assign values through slices*

In [12]: `s1`

Out[12]: `array([ 1, 4, 9, 99, 25])`

Note that the above assignment will also change value in array `v1`. `v1` and `s1` point to the same array in memory

```
In [13]: v1
```

```
Out[13]: array([ 4,  9, 99])
```

```
In [14]: v1[0] # first element of an array called by index 0, NOT 1
```

```
Out[14]: np.int64(4)
```

```
In [15]: v1[0] = 200
```

```
In [16]: v1
```

```
Out[16]: array([200,  9, 99])
```

```
In [17]: s1
```

```
Out[17]: array([ 1, 200,  9, 99, 25])
```

```
In [18]: print(s1.dtype) # dtype method displays the element's class;  
         # s1 is a one-dimensional array  
         # type of elements is integer (int32)  
         print(s1.shape )
```

```
int64  
(5,)
```

```
In [19]: type(s1) # function type displays the data structure or class: numeric array
```

```
Out[19]: numpy.ndarray
```

```
In [20]: s2 #to see the array you could also print s1
```

```
Out[20]: array([[ 1,  2,  3,  4,  5],  
                [ 1,  4,  9, 16, 25],  
                [ 1,  8, 27, 64, 125]])
```

```
In [21]: print (s2)
```

```
[[ 1  2  3  4  5]  
 [ 1  4  9 16 25]  
 [ 1  8 27 64 125]]
```

```
In [22]: s2[2]  
         # For a matrix, inputting a single index pulls out the corresponding row  
         # (here 2 means the 3rd row from the top)
```

```
Out[22]: array([ 1,  8, 27, 64, 125])
```

```
In [23]: s2[:,2] #to extract a specific column
```

```
Out[23]: array([ 3,  9, 27])
```

If you want to select an entire row, # (number of row) followed by :. Note, if you have a matrix and want to be more precise (or to pick a column), write : in the non-prescribed row/column.

```
In [24]: s2[1:] # call all rows 1 and below
```

```
Out[24]: array([[ 1,  4,  9, 16, 25],
               [ 1,  8, 27, 64, 125]])
```

```
In [25]: s2[1:,:] # Here we explicitly indicate that we want all columns
```

```
Out[25]: array([[ 1,  4,  9, 16, 25],
               [ 1,  8, 27, 64, 125]])
```

```
In [26]: s2[:1] # slice array from the first row, index 0, to the row with index 1
          # row with index 1 not included.
```

```
Out[26]: array([[1, 2, 3, 4, 5]])
```

```
In [27]: s2[:2] # slice array from the first row, index 0, to the row with index 2
          # row with index 2 not included.
```

```
Out[27]: array([[ 1,  2,  3,  4,  5],
               [ 1,  4,  9, 16, 25]])
```

```
In [28]: s2[1,3] # call element in row with index 1, column with index 3
```

```
Out[28]: np.int64(16)
```

```
In [29]: s2 #let's print s2 again so that we can see it
```

```
Out[29]: array([[ 1,  2,  3,  4,  5],
               [ 1,  4,  9, 16, 25],
               [ 1,  8, 27, 64, 125]])
```

```
In [30]: s2[:,::2] # all columns with even indices
```

```
Out[30]: array([[ 1,  3,  5],
               [ 1,  9, 25],
               [ 1, 27, 125]])
```

```
In [31]: s2[:,::-1] # flips the order of the columns -- columns displayed backwards
```

```
Out[31]: array([[ 5,  4,  3,  2,  1],
               [25, 16,  9,  4,  1],
               [125, 64, 27,  8,  1]])
```

```
In [32]: (s2[0])[::-1] # selects 0th row, flips the order of elements
```

```
Out[32]: array([5, 4, 3, 2, 1])
```

Addition and multiplication act element by element for arrays of equal dimensions

```
In [33]: s1
```

```
Out[33]: array([ 1, 200,  9, 99, 25])
```

```
In [34]: s3
```

```
Out[34]: array([ 2,  5, 10, 17, 26])
```

```
In [35]: s1+s3
```

```
Out[35]: array([ 3, 205, 19, 116, 51])
```

```
In [36]: s1*s3
```

```
Out[36]: array([  2, 1000,  90, 1683, 650])
```

Broadcasting

Broadcasting allows us to add lower-dimensional arrays to higher-dimensional arrays

```
In [37]: s1
```

```
Out[37]: array([ 1, 200,  9, 99, 25])
```

```
In [38]: s2
```

```
Out[38]: array([[ 1,  2,  3,  4,  5],
                [ 1,  4,  9, 16, 25],
                [ 1,  8, 27, 64, 125]])
```

```
In [39]: s1 + s2 # s1 will be broadcast and added to each "row" in s2; s1 is 1x5 and s2 is 3
# we will often be adding a vector to each row or column in a matrix
```

```
Out[39]: array([[ 2, 202, 12, 103, 30],
                [ 2, 204, 18, 115, 50],
                [ 2, 208, 36, 163, 150]])
```

```
In [40]: # a single column matrix with 3 rows
s3 = np.array([[1],[2],[0]])
s3
```

```
Out[40]: array([[1],
                [2],
                [0]])
```

```
In [41]: # added to 3x4 matrix
s2+s3
```

```
Out[41]: array([[ 2,  3,  4,  5,  6],
                [ 3,  6, 11, 18, 27],
                [ 1,  8, 27, 64, 125]])
```

Logical operations

In [42]: `s2`

Out[42]: `array([[1, 2, 3, 4, 5],
[1, 4, 9, 16, 25],
[1, 8, 27, 64, 125]])`

In [43]: `s2 > 10` *# boolean selection array; turns numerical array into logical array*

Out[43]: `array([[False, False, False, False, False],
[False, False, False, True, True],
[False, False, True, True, True]])`

In [44]: `s1`

Out[44]: `array([ 1, 200, 9, 99, 25])`

In [45]: `s1 > 10` *# creates an array of true/false indices*

Out[45]: `array([False, True, False, True, True])`

In [46]: `s1[s1>10]` *# fancy indexing using boolean selection array:
preserve elements of s1 that are greater than 10
outputs numerical array (as opposed to above)*

Out[46]: `array([200, 99, 25])`

In [47]: `s2`

Out[47]: `array([[1, 2, 3, 4, 5],
[1, 4, 9, 16, 25],
[1, 8, 27, 64, 125]])`

In [48]: `s2 >= s1` *# logical broadcasting:
takes s2 row by row and compares every row in s2 with s1*

Out[48]: `array([[True, False, False, False, False],
[True, False, True, False, True],
[True, False, True, False, True]])`

In [49]: `s2[s2 >= s1]`
*#takes the matrix above and keeps numbers in position with True values
note that it scans row by row and produces a row of values*

Out[49]: `array([ 1, 1, 9, 25, 1, 27, 125])`

In [50]: `s1`

Out[50]: `array([ 1, 200, 9, 99, 25])`

```
In [51]: np.where(s1 > 10, 5, -2) # where function in Numpy select values as well.
# vectorized conditional: all values greater than 10, replace with 5,
# others, i.e., value less than 10, replace with -2
```

```
Out[51]: array([-2,  5, -2,  5,  5])
```

```
In [52]: np.where(s1 > 10, 'big', 'small') # where clause.
# vectorized conditional: all values greater than 10, replace with 'big,'
# values less than 10, replace with 'small'.
# This turns numeric array into a string array.
```

```
Out[52]: array(['small', 'big', 'small', 'big', 'big'], dtype='<U5')
```

```
In [53]: np.where(s2 % 2 == 0, 'even', 'odd') # module division --modulus,
# if the remainder of dividing s2[] by 2 is 0, write even, if 1, write odd
```

```
Out[53]: array([[ 'odd', 'even', 'odd', 'even', 'odd'],
                [ 'odd', 'even', 'odd', 'even', 'odd'],
                [ 'odd', 'even', 'odd', 'even', 'odd']], dtype='<U4')
```

```
In [54]: np.where(s2 > 10, 10, s2)
#if greater than 10 then print 10, if smaller than 10 then print value
```

```
Out[54]: array([[ 1,  2,  3,  4,  5],
                [ 1,  4,  9, 10, 10],
                [ 1,  8, 10, 10, 10]])
```

Numpy arrays are richer data structure than simple numerical matrices: can have logical arrays, string arrays, etc

Basic Linear Algebra

Important basic operations:

Identity matrix

$$\mathbf{I} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

```
In [55]: np.identity(3)
#identity function generates identity matrix of desired dimension
```

```
Out[55]: array([[1., 0., 0.],
                [0., 1., 0.],
                [0., 0., 1.]])
```



```
In [56]: np.identity(5)
```

```
Out[56]: array([[1., 0., 0., 0., 0.],
               [0., 1., 0., 0., 0.],
               [0., 0., 1., 0., 0.],
               [0., 0., 0., 1., 0.],
               [0., 0., 0., 0., 1.]])
```

Vector addition:

$$\mathbf{s}_1 = (a_1, b_1, c_1, d_1) = (1, 200, 9, 99, 25)$$

$$\mathbf{s}_3 = (a_3, b_3, c_3, d_3) = (2, 5, 10, 17, 26)$$

$$\mathbf{s}_1 + \mathbf{s}_3 = (a_1, b_1, c_1, d_1) + (a_3, b_3, c_3, d_3) = (a_1 + a_3, b_1 + b_3, c_1 + c_3, d_1 + d_4) = (3, 205, 19, 116, 51)$$



```
In [57]: s1 = np.array([1, 200, 9, 99, 25])
         s2 = np.array([2, 5, 10, 17, 26])
```

```
In [58]: s1+s2 # adds arrays element by element
```

```
Out[58]: array([ 3, 205, 19, 116, 51])
```

Multiplication between a scalar and a vector:

$$\beta \mathbf{A} = \beta(a_1, a_2, a_3) = (\beta a_1, \beta a_2, \beta a_3)$$

$$\beta = 2$$

$$\mathbf{s}_1 = (1, 200, 9, 99, 25)$$

$$\beta \mathbf{s}_1 = (2, 400, 18, 198, 50)$$

```
In [59]: s1
```

```
Out[59]: array([ 1, 200,  9,  99, 25])
```

```
In [60]: a=2;
         a*s1
```

```
Out[60]: array([ 2, 400, 18, 198, 50])
```

Element-wise multiplication:

$$\sum_{i=1}^n s_{1,i} s_{3,i} \hat{\mathbf{x}}_i = (2, 1000, 90, 1643, 650)$$

In [61]: `s1`

Out[61]: `array([ 1, 200, 9, 99, 25])`

In [62]: `s3 = np.array([2,0.1,10,3,26])`

In [63]: `s1*s3 # multiplies elmenet by element`

Out[63]: `array([ 2., 20., 90., 297., 650.])`

In [64]: `s2`

Out[64]: `array([ 2, 5, 10, 17, 26])`

In [65]: `s1*s2 #broadcasting multiplication: s1 multiplies every row in s2`

Out[65]: `array([ 2, 1000, 90, 1683, 650])`

Determinant

The determinant is a useful quantity by which we can characterize square matrices

$$\det \begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc$$

For a general $m \times m$ matrix one can use the Leibniz formula (see wiki or MathWorld)

In [66]: `mm = np.random.randn(5,5)`
`mm`

Out[66]: `array([[0.30227016, -0.20483772, 0.30971865, -0.68965521, 0.22567806],
[1.25766137, -1.55870632, -0.69657508, -1.85337473, 0.27672373],
[-0.7991052 , 1.21851364, 2.1231061 , 0.50639747, -0.75630739],
[-0.95055175, 0.75961924, -0.7633847 , 0.17951362, -0.19752568],
[1.99103237, -0.15529655, -0.62723104, -0.19182101, 0.5590344]])`

In [67]: `np.linalg.det(mm) #finds determinant; Numpy has linear algebra subpackage`

Out[67]: `np.float64(-2.459205929374109)`

Transpose:

$$(\mathbf{A}^T)_{i,j} = \mathbf{A}_{j,i}$$

Element ij of a transpose matrix is element ji of the original matrix; note the flip in the indices

In [68]: `s1`

Out[68]: `array([ 1, 200, 9, 99, 25])`

In [69]: `np.transpose(s1) #recall from lecture that this transpose is a vertical vector`

Out[69]: `array([ 1, 200, 9, 99, 25])`

In [70]: `# Numpy does not automatically display the transpose vector as vertical,
You can do so by using reshape.
s1vertical = s1.reshape(5,1)`

In [71]: `s1vertical`

Out[71]: `array([[1],
[200],
[9],
[99],
[25]])`

In [72]: `# Go back to the original vector
np.transpose(s1vertical)`

Out[72]: `array([[ 1, 200, 9, 99, 25]])`

In [73]: `# With rectangular matrices the effects of transpose are easier to see
s2 = np.array([[1,2,3,4,5],
[1,4,9,16,25],
[1,8,27,64,125]]) # array of three rows and five columns`

In [74]: `np.transpose(s2) # with matrices you clearly see the transpose`

Out[74]: `array([[1, 1, 1],
[2, 4, 8],
[3, 9, 27],
[4, 16, 64],
[5, 25, 125]])`

Inverse:

for a square matrix with non-zero determinant, we can take the inverse, which is defined for a 2x2 matrix as:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

a general $m \times n$ matrix can be inverted by numeric methods such as Gauss-Jordan elimination, Gaussian elimination, or LU decomposition. Numpy does it automatically for us.

Note the determinant in the denominator

```
In [75]: s4 = np.array([[1,2],
                      [1,4]])
```

```
In [76]: s4inv = np.linalg.inv(s4) # here's the linear algebra subpackage again;
        # we want the inverse matrix
        print(s4inv)
```

```
[[ 2.  -1. ]
 [-0.5  0.5]]
```

```
In [77]: # Element by element multiplication does not produce a unity matrix
        s4*s4inv
```

```
Out[77]: array([[ 2. , -2. ],
                [-0.5,  2. ]])
```

```
In [78]: # np.matmul() function performs proper matrix multiplication
        s44 = np.matmul(s4,s4inv)
```

```
In [79]: print(s44)
```

```
[[1. 0.]
 [0. 1.]]
```

```
In [80]: s5 = np.array([[1,2,3,4],
                      [1,4,9,16],
                      [1,8,27,64],
                      [1,16,81,256]])
        s6 = np.linalg.inv(s5) #s6 is the inverse
        s6
```

```
Out[80]: array([[ 4.          , -4.33333333,  1.5          , -0.16666667],
                [-3.          ,  4.75         , -2.          ,  0.25         ],
                [ 1.33333333, -2.33333333,  1.16666667, -0.16666667],
                [-0.25         ,  0.45833333, -0.25         ,  0.04166667]])
```

Matrix multiplicaiton in Numpy

```
In [81]: s7 = np.matmul (s5,s6) #multiplying the two matrices
        print (s7)
        # note 1-s on the diagonal. All elements outside diagonal are very close to 0.
        #This is just error of numeric calculation
```

```
[[ 1.00000000e+00 -2.22044605e-16 -4.44089210e-16 -5.55111512e-17]
 [ 1.33226763e-15  1.00000000e+00 -2.66453526e-15 -1.11022302e-16]
 [-3.55271368e-15  1.77635684e-14  1.00000000e+00  8.88178420e-16]
 [ 1.42108547e-14  1.42108547e-14  0.00000000e+00  1.00000000e+00]]
```

The inner (dot) and outer products are defined by the dot and outer functions

Dot product:

In general

$$\mathbf{A} \cdot \mathbf{B} = \mathbf{A}^T \mathbf{B} = \begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} = a_1 b_1 + a_2 b_2 + a_3 b_3$$

Or for our vectors $\mathbf{s}_1$ and $\mathbf{s}_3$

$$\mathbf{s}_1 \cdot \mathbf{s}_3 = \begin{bmatrix} 1 & 200 & 9 & 99 & 25 \end{bmatrix} \begin{bmatrix} 2 \\ 5 \\ 10 \\ 17 \\ 26 \end{bmatrix} = 2 + 1000 + 90 + 1683 + 650 = 3425$$

The dot product gives you a scalar

```
In [82]: s1
```

```
Out[82]: array([ 1, 200,  9, 99, 25])
```

```
In [83]: s3
```

```
Out[83]: array([ 2. ,  0.1, 10. ,  3. , 26. ])
```

```
In [84]: np.dot(s1,s3) # dot product -- dot product of two vectors is a scalar
```

```
Out[84]: np.float64(1059.0)
```

Outer product

$$\mathbf{A} \otimes \mathbf{B} = \mathbf{A} \mathbf{B}^T = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} \begin{bmatrix} b_1 & b_2 & b_3 \end{bmatrix} = \begin{bmatrix} a_1 b_1 & a_1 b_2 & a_1 b_3 \\ a_2 b_1 & a_2 b_2 & a_2 b_3 \\ a_3 b_1 & a_3 b_2 & a_3 b_3 \end{bmatrix}$$

```
In [85]: np.outer(s1,s3) # matrix multiplication or outer product (or cartesian product)
```

```
Out[85]: array([[2.000e+00, 1.000e-01, 1.000e+01, 3.000e+00, 2.600e+01],
                [4.000e+02, 2.000e+01, 2.000e+03, 6.000e+02, 5.200e+03],
                [1.800e+01, 9.000e-01, 9.000e+01, 2.700e+01, 2.340e+02],
                [1.980e+02, 9.900e+00, 9.900e+02, 2.970e+02, 2.574e+03],
                [5.000e+01, 2.500e+00, 2.500e+02, 7.500e+01, 6.500e+02]])
```

Cross product

An additional product that is valid for vectors that are 3-elements in length is the cross product (a more generalized form is the outer product).

$$\mathbf{A} \times \mathbf{B} = (a_1, a_2, a_3) \times (b_1, b_2, b_3) = (a_2b_3 - a_3b_2, a_3b_1 - a_1b_3, a_1b_2 - a_2b_1)$$

Length/norm of the cross product is the area of the rhombus defined by vectors s_1 and s_3 .
The direction of that vector is orthogonal to the plane defined by vectors s_1 and s_3

```
In [86]: s6 = np.array([1,2,3])
s7 = np.array([4,5,6])

np.cross(s6,s7)
```

```
Out[86]: array([-3,  6, -3])
```

Special kinds of matrices and vectors

The following assumes that you are working with square matrices, i.e. $m = n$

Diagonal matrices

$$\mathbf{A} = \begin{bmatrix} a_1 & 0 & 0 \\ 0 & a_2 & 0 \\ 0 & 0 & a_3 \end{bmatrix}$$

Where $A_{i,j} = 0$ if $i \neq j$

Symmetric matrices

A matrix is symmetric if it is equal to its transpose, i.e.:

$$\mathbf{A} = \mathbf{A}^T$$

```
In [87]: s8 = np.array([[1,7,3], # a symmetric matrix. The transpose will equal the original
                        [7,4,-5],
                        [3,-5,6]])

np.transpose(s8)
```

```
Out[87]: array([[ 1,  7,  3],
                [ 7,  4, -5],
                [ 3, -5,  6]])
```

Orthonormal vectors

A collection of real vectors $v_1, v_2, v_3, \text{etc.}$, is orthonormal if

- vectors have unit norm $\|a_i\| = 1$

- they are mutually orthogonal: $a_i^T a_j = 0$ if $i \neq j$
- the square of the norm is equal to the dot product of the vector with itself

```
In [88]: e1 = np.array([0,0,-1]) # z-coordinate is -1
e2 = np.array([1/np.sqrt(2),1/np.sqrt(2),0])
#two orthonormal vectors; x and y-coordinates are 1/(square root of 2);norm
e3 = 1/np.sqrt(2)*np.array([1,-1,0])
#e2 and e3 are both in the xy plane; e3 is e2 rotated by 90 degrees
```

e1, e2 and e3 are three mutually orthonormal vectors because:

```
In [89]: np.dot(e1,e2)
```

```
Out[89]: np.float64(0.0)
```

```
In [90]: np.dot(e2,e3)
```

```
Out[90]: np.float64(0.0)
```

```
In [91]: np.dot(e3,e1)
```

```
Out[91]: np.float64(0.0)
```

We prove that the vectors are normal when their dot product is zero. You can think of these three vectors as the new basis vectors for the three dimensional space

Orthogonal matrices

A matrix is orthogonal if it is a square matrix whose rows are mutually orthonormal and whose columns are mutually orthonormal:

$$\mathbf{A}^T \mathbf{A} = \mathbf{A} \mathbf{A}^T = \mathbf{I}$$

This implies that $\mathbf{A}^{-1} = \mathbf{A}^T$

```
In [92]: m1 = np.array([[0,1,0,0],
                        [0,0,0,1],
                        [1,0,0,0],
                        [0,0,1,0]])

m2 = np.array([[0,0,1,0],
                [1,0,0,0],
                [0,0,0,1],
                [0,1,0,0]])

np.dot(m1,m2)
```

```
Out[92]: array([[1, 0, 0, 0],
                [0, 1, 0, 0],
                [0, 0, 1, 0],
                [0, 0, 0, 1]])
```

Eigen-decomposition

Good discussion of this in ch2 of the Deep Learning book we provided. Often, we can better understand mathematical objects by breaking them into constituent parts. An example of this is when you decompose a number into its prime factors, for example $12 = 2 \times 2 \times 3$. In a similar way, decomposing a matrix into its eigenvalues and eigenvectors can give us information about a matrix that is not obvious from its representation as an array.

An eigenvector of a square matrix $\mathbf{A}$ is a non-zero vector $\mathbf{v}$ such that multiplication by $\mathbf{A}$ alters only the scale of $\mathbf{v}$:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$$

The scalar λ is known as the eigenvalue corresponding to the eigenvector

```
In [93]: A = np.array( [[4,0,1],
                       [2,-2,3],
                       [7,5,0]])

v = np.array([1,1,2])

np.dot(A,v)
```

```
Out[93]: array([ 6,  6, 12])
```

```
In [94]: 6*v    # so we can see that the eigenvalue for v is 6;
```

```
Out[94]: array([ 6,  6, 12])
```

```
In [95]: dm = np.diag((1,2,3)) #create a diagonal matrix with 1,2,3 along the diagonal
```

```
In [96]: dm
```

```
Out[96]: array([[1, 0, 0],
                [0, 2, 0],
                [0, 0, 3]])
```

```
In [97]: np.linalg.eig(dm) # eig function calculates eigenvectors;
# creates an array with the eigenvalues in the first row
# [1., 2., 3.] are the eigenvalues.
# But it also gives you a matrix with the eigenvectors, in which the first row cont
# the eigenvalues and the second
# and remaining rows are the eigenvectors. In other words, this is an array with 2
# the first contains a vector of eigenvalues and the second contains the eigenvecto
```

```
Out[97]: EigResult(eigenvalues=array([1., 2., 3.]), eigenvectors=array([[1., 0., 0.],
                               [0., 1., 0.],
                               [0., 0., 1.])))
```



```
In [98]: np.dot(dm,[1,0,0])  
# dot products of the diagonal matrix and every eigenvector will give us the eigenv  
  
Out[98]: array([1, 0, 0])  
  
In [99]: np.dot(dm,[0,1,0])  
  
Out[99]: array([0, 2, 0])  
  
In [100]: np.dot(dm,[0,0,1])  
  
Out[100]: array([0, 0, 3])
```

Pandas

Pandas is a software library for Python for data manipulation and analysis with an emphasis on operations on numerical tables. Several attractive and popular features of pandas:

- Robust data input/output capabilities with integrated indexing
- Powerful API on top of NumPy `ndarray` data structure
- Analog to R DataFrame
- Versatile data visualization capabilities: matrix or database table.
- Specific characteristic:
 - Pandas are similar to R DataFrames and Excel file.
 - Data alignment and integrated handling of missing data.
 - Reshaping and pivoting of data sets.
 - Label-based slicing, fancy indexing,[3] and subsetting of large data sets.
 - Data structure column insertion and deletion.
 - Group by engine allowing split-apply-combine operations on data sets.
 - Data set merging and joining.
 - Hierarchical axis indexing to work with high-dimensional data in a lower-dimensional data structure.
 - Time series-functionality:
 - SQL like syntax for extraction of rows and columns

Supported data formats:

- CSV
- Excel
- SQL
- JSON
- HDF5
- pickle

- msgpack
- Stata
- BigQuery

To show what Pandas can do we will look at data on Latin American politicians contained in a text file

In [101... `!pip install pandas`

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pandas in c:\users\mleen\appdata\roaming\python\python313\site-packages (2.3.2)
Requirement already satisfied: numpy>=1.26.0 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from pandas) (2.3.2)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from pandas) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
[notice] A new release of pip is available: 25.2 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [102... `import sys`
`import numpy as np`
`import pandas as pd`

`pd.options.display.max_rows = 20`
`pd.options.display.max_columns = 15`
`pd.__version__`

Out[102... `'2.3.2'`

In [103... `import sys` *#system package that allows you to extract info about the computer itse*
This module provides access to some variables used or maintained by the interpret
and to functions that interact strongly with the interpreter. It is always availa

In [104... `sys.path`

Out[104... `['c:\\Python313\\python313.zip',`
`'c:\\Python313\\DLLs',`
`'c:\\Python313\\Lib',`
`'c:\\Python313',`
`'',`
`'C:\\Users\\mleen\\AppData\\Roaming\\Python\\Python313\\site-packages',`
`'C:\\Users\\mleen\\AppData\\Roaming\\Python\\Python313\\site-packages\\win32',`
`'C:\\Users\\mleen\\AppData\\Roaming\\Python\\Python313\\site-packages\\win32\\li`
`b',`
`'C:\\Users\\mleen\\AppData\\Roaming\\Python\\Python313\\site-packages\\Pythonwi`
`n',`
`'c:\\Python313\\Lib\\site-packages']`

```
In [105... sitepkgs = r"C:\Users\mleen\michael_sandbox\metcs599\labs\lab1"
#file in my directory
testfile = "utf16_ex.txt" # file contains data on politicians from various countries
```

```
In [106... import os

path = os.path.join(sitepkgs, testfile)
```

```
In [107... data = pd.read_csv(path, sep="\t", encoding="utf-16", parse_dates=["Date"])
# reads the data (like an excel spread sheet -- first row has column names, Pandas
# \t means it separates by tab: sep declares elements are separated;
# encoding declares what sort of unicode to use;
# and parse_dates deals with the dates (Pandas is pretty good at recognizing dates.
```

```
C:\Users\mleen\AppData\Local\Temp\ipykernel_34380\1315178558.py:1: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  data = pd.read_csv(path, sep="\t", encoding="utf-16", parse_dates=["Date"])
```

```
In [108... data # native to pandas; prints the data
```

Out[108...

	Country	State/City	Title	Date	Catalogue	Wikipedia Election Page	Wikipedia Individual Page	Electoral Institution in Country
0	Venezuela	NaN	President	2012-10-07	Hugo Rafael Chavez Frias	Hugo Chávez	Hugo Chávez	Hugo Chavez
1	Venezuela	NaN	President	2012-10-07	Henrique Capriles Radonski	Henrique Capriles Radonski	Henrique Capriles Radonski	Henrique Capriles Radonski
2	Venezuela	NaN	President	2012-10-07	Maria Josefina Bolivar	María Bolívar	María Bolívar	Maria Bolivar
3	Venezuela	NaN	President	2012-10-07	Orlando Chirino	Orlando Chirinos	None.	Orlando Chirino
4	Venezuela	NaN	President	2012-10-07	Luis Reyes	Luis Reyes Castillo	None.	Luis Reyes
...	...	...	...	...	...	...	...	...
45	NaN	NaN	NaN	NaT	NaN	NaN	NaN	NaN
46	NaN	NaN	NaN	NaT	NaN	NaN	NaN	NaN
47	NaN	NaN	NaN	NaT	NaN	NaN	NaN	NaN
48	NaN	NaN	NaN	NaT	NaN	NaN	NaN	NaN
49	NaN	NaN	NaN	NaT	NaN	NaN	NaN	NaN

50 rows × 11 columns



In [109...

```
data.dropna(subset=['Country', 'Date', 'Title'], inplace=True) # remove objects with
# drops rows 43 and on. If anywhere there's an NaN in those columns, it will drop them
# Pandas can drop rows that have non-defined values, or you can replace those values with
# specified values
```

In [110...

data

Out[110...

	Country	State/City	Title	Date	Catalogue	Wikipedia Election Page	Wikipedia Individual Page	Electoral Institution in Country
0	Venezuela	NaN	President	2012-10-07	Hugo Rafael Chavez Frias	Hugo Chávez	Hugo Chávez	Hugo Chavez
1	Venezuela	NaN	President	2012-10-07	Henrique Capriles Radonski	Henrique Capriles Radonski	Henrique Capriles Radonski	Henrique Capriles Radonski
2	Venezuela	NaN	President	2012-10-07	Maria Josefine Bolivar	María Bolívar	María Bolívar	Maria Bolivar
3	Venezuela	NaN	President	2012-10-07	Orlando Chirino	Orlando Chirinos	None.	Orlando Chirino
4	Venezuela	NaN	President	2012-10-07	Luis Reyes	Luis Reyes Castillo	None.	Luis Reyes
...	...	...	...	...	...	...	...	...
38	Ecuador	NaN	President	2013-02-07	Lucio Edwin Gutiérrez Borbúa	Lucio Gutiérrez	Lucio Gutiérrez	Lucio Gutiérrez
39	Ecuador	NaN	President	2013-02-07	Álvaro Fernando Noboa Pontón	Álvaro Noboa	Álvaro Noboa	Alvaro Noboa
40	Ecuador	NaN	President	2013-02-07	Alberto Acosta Espinosa	Alberto Acosta Espinosa	Alberto Acosta Espinosa	Alberto Acosta
41	Ecuador	NaN	President	2013-02-07	Mauricio Rodas Espinel	Mauricio Rodas	Mauricio Rodas	Mauricio Rodas
42	Ecuador	NaN	President	2013-02-07	Norman Wray	Norman Wray	None.	Norman Wray

43 rows × 11 columns



In [111...

```
!pip install xlwt
# Library to write older version of Excel file
```

Defaulting to user installation because normal site-packages is not writeable
 Requirement already satisfied: xlwt in c:\users\mleen\appdata\roaming\python\python313\site-packages (1.3.0)

```
[notice] A new release of pip is available: 25.2 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [112... `!pip install openpyxl`

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: openpyxl in c:\users\mleen\appdata\roaming\python\python313\site-packages (3.1.5)
Requirement already satisfied: et-xmlfile in c:\users\mleen\appdata\roaming\python\python313\site-packages (from openpyxl) (2.0.0)
```

```
[notice] A new release of pip is available: 25.2 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [113... `data.to_excel('people.xlsx')`
exporting function specific to pandas -- takes your data set and exports it into

In [114... `data[data.Country == 'Chile']` *#selecting data for Chile only*

Out[114...

	Country	State/City	Title	Date	Catalogue	Wikipedia Election Page	Wikipedia Individual Page	Electoral Institution in Country	Tw
6	Chile	Santiago	Mayor	2012-10-28	Carolina Montserrat Toha Morales	Carolina Tohá Morales	Carolina Tohá	Carolina Toha Morales	Car
7	Chile	Santiago	Mayor	2012-10-29	Pablo Zalaquett Said	Pablo Zalaquett Said	Pablo Zalaquett	Pablo Zalaquett Said	P Zalac
8	Chile	Santiago	Mayor	2012-10-30	Waldo Mora Longa	Waldo Mora Longa	None.	Waldo Mora Longa	W Mc
9	Chile	Santiago	Mayor	2012-10-31	Mario Aguilar	Mario Aguilar Arevalo	None.	Mario Aguilar Arevalo	N Ag
10	Chile	Santiago	Mayor	2012-11-01	Ivan Carrasco	Iván Carrasco Mora	None.	Ivan Carrasco Mora	N

In [115... `data[data.Date < '2012-11']` *#selecting dates before November 2012*

Out[115...

	Country	State/City	Title	Date	Catalogue	Wikipedia Election Page	Wikipedia Individual Page	Electoral Institution in Country	
0	Venezuela	NaN	President	2012-10-07	Hugo Rafael Chavez Frias	Hugo Chávez	Hugo Chávez	Hugo Chavez	
1	Venezuela	NaN	President	2012-10-07	Henrique Capriles Radonski	Henrique Capriles Radonski	Henrique Capriles Radonski	Henrique Capriles Radonski	H
2	Venezuela	NaN	President	2012-10-07	Maria Josefina Bolivar	María Bolívar	María Bolívar	Maria Bolivar	
3	Venezuela	NaN	President	2012-10-07	Orlando Chirino	Orlando Chirinos	None.	Orlando Chirino	C
4	Venezuela	NaN	President	2012-10-07	Luis Reyes	Luis Reyes Castillo	None.	Luis Reyes	
5	Venezuela	NaN	President	2012-10-07	Reina Sequera	Reina Sequera	Reina Sequera	Reina Sequera	S
6	Chile	Santiago	Mayor	2012-10-28	Carolina Montserrat Toha Morales	Carolina Tohá Morales	Carolina Tohá	Carolina Toha Morales	C
7	Chile	Santiago	Mayor	2012-10-29	Pablo Zalaquett Said	Pablo Zalaquett Said	Pablo Zalaquett	Pablo Zalaquett Said	Za
8	Chile	Santiago	Mayor	2012-10-30	Waldo Mora Longa	Waldo Mora Longa	None.	Waldo Mora Longa	
9	Chile	Santiago	Mayor	2012-10-31	Mario Aguilar	Mario Aguilar Arevalo	None.	Mario Aguilar Arevalo	

In [116... `data.Country.unique()` *#select unique country names without repetition*Out[116... `array(['Venezuela', 'Chile', 'Nicaragua', 'Ecuador'], dtype=object)`In [117... `data.Country` *#this is the entire Country column, and you can see repetition*

```
Out[117... 0    Venezuela
          1    Venezuela
          2    Venezuela
          3    Venezuela
          4    Venezuela
          ...
         38    Ecuador
         39    Ecuador
         40    Ecuador
         41    Ecuador
         42    Ecuador
Name: Country, Length: 43, dtype: object
```

```
In [118... data.Country.value_counts()
```

```
Out[118... Country
Venezuela    25
Ecuador      7
Nicaragua    6
Chile        5
Name: count, dtype: int64
```

Matplotlib

Matplotlib is a Python 2D plotting library that is both approachable but still very capable of generating sophisticated images.

- Images are MATLAB-like for simple plotting
 - MATLAB-like features make matplotlib images easily incorporated into industry standards
- Default plot styles allow for rapid visualization of data
- Deep integration with Python libraries
- Reliant on other Python libraries such as NumPy

We will look at how to display data and functions

```
In [119... !pip install matplotlib
```


Defaulting to user installation because normal site-packages is not writeable
 Requirement already satisfied: matplotlib in c:\users\mleen\appdata\roaming\python\python313\site-packages (3.10.6)
 Requirement already satisfied: contourpy>=1.0.1 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from matplotlib) (1.3.3)
 Requirement already satisfied: cyclor>=0.10 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from matplotlib) (0.12.1)
 Requirement already satisfied: fonttools>=4.22.0 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from matplotlib) (4.59.2)
 Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from matplotlib) (1.4.9)
 Requirement already satisfied: numpy>=1.23 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from matplotlib) (2.3.2)
 Requirement already satisfied: packaging>=20.0 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from matplotlib) (25.0)
 Requirement already satisfied: pillow>=8 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from matplotlib) (11.3.0)
 Requirement already satisfied: pyparsing>=2.3.1 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from matplotlib) (3.2.3)
 Requirement already satisfied: python-dateutil>=2.7 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from matplotlib) (2.9.0.post0)
 Requirement already satisfied: six>=1.5 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
 [notice] A new release of pip is available: 25.2 -> 25.3
 [notice] To update, run: python.exe -m pip install --upgrade pip

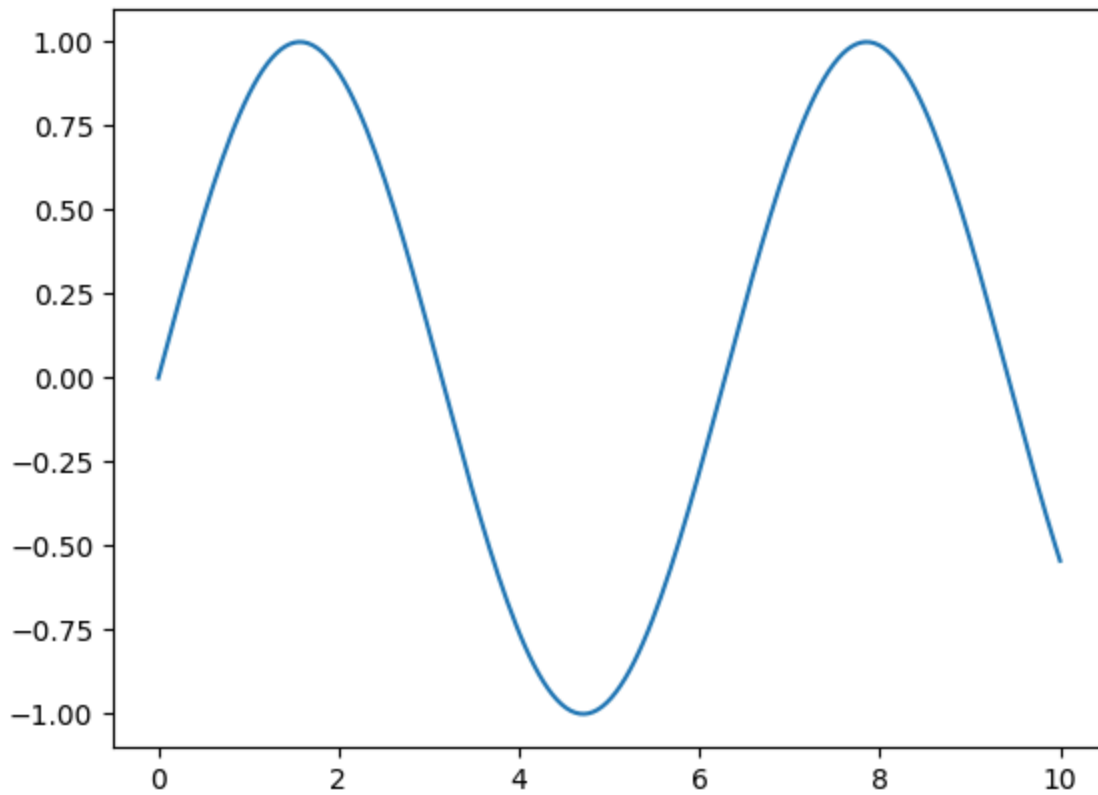
In [120... `%matplotlib inline`
*# magic function, says that matplotlib will be included in IPython and makes output
 # and be stored in the notebook
 # important because you want your graphics in the jupyter notebook*

In [121... `import numpy as np`
`from pylab import *`

`n = 200` *# number of elements in array*
`x = np.linspace(0,10,n)` *# creates a linear array of 200 elements between 0 and 1
 # (can add additional term to define step size)*
`data = np.sin(x)` *#for every element, calculate the sin*
*# note that unlike in mathematics, sin function accepts vector as its argument,
 # and calculates sin for every element of this vector.*
*# You can also do this with a for loop that passes each scalar element in the vector
 # however the for loop is much slower than passing a vector to the sin() function*
`noise = np.random.randn(n)`
*# random normal distribution (generates values that are normally distributed around
 # so you get positive and negative values) ; adding noise to the sin function;
 # noise is a separate array*
`noise2 = np.random.normal(0,.1,n)`
0 is the mean or center of the distribution, .1 is the standard deviation

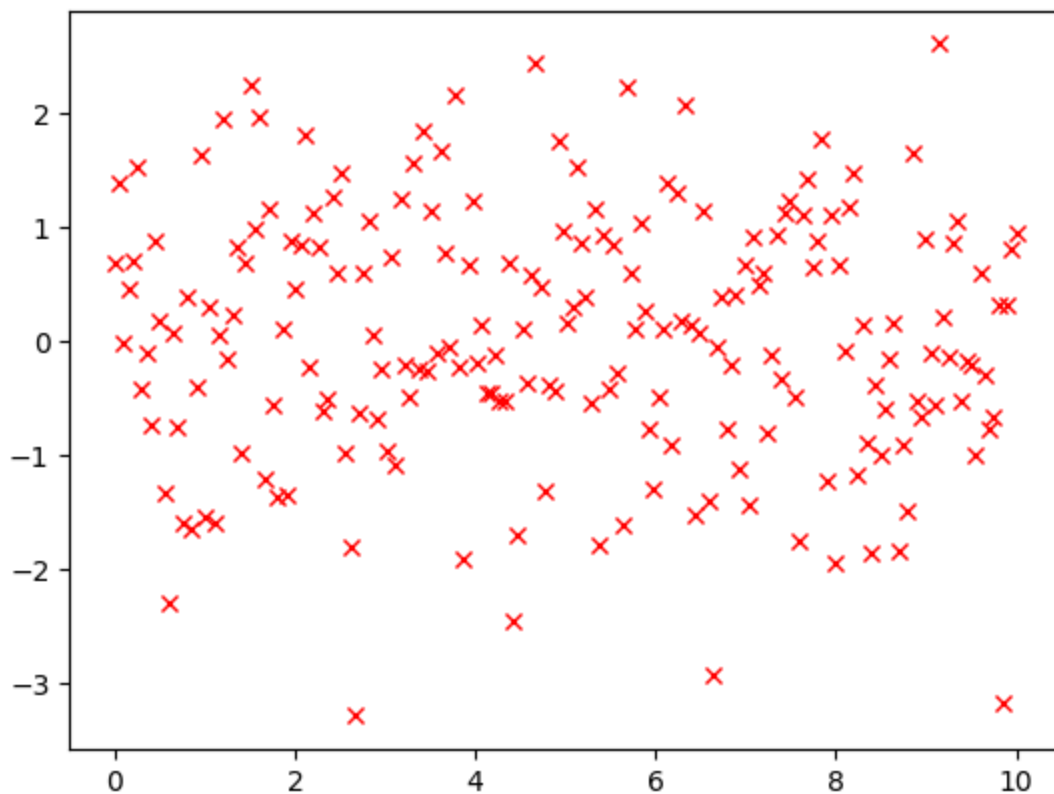
In [122... `plot(x,data)` *# here we plot the sin() function*

Out[122... `[<matplotlib.lines.Line2D at 0x271ceef91d0>]`



In [123... `plot(x,noise, 'rx')` # here we plot the noise; note that the data points are dense a
which means the function does have a normal distribution

Out[123... [`<matplotlib.lines.Line2D at 0x271cf166710>`]

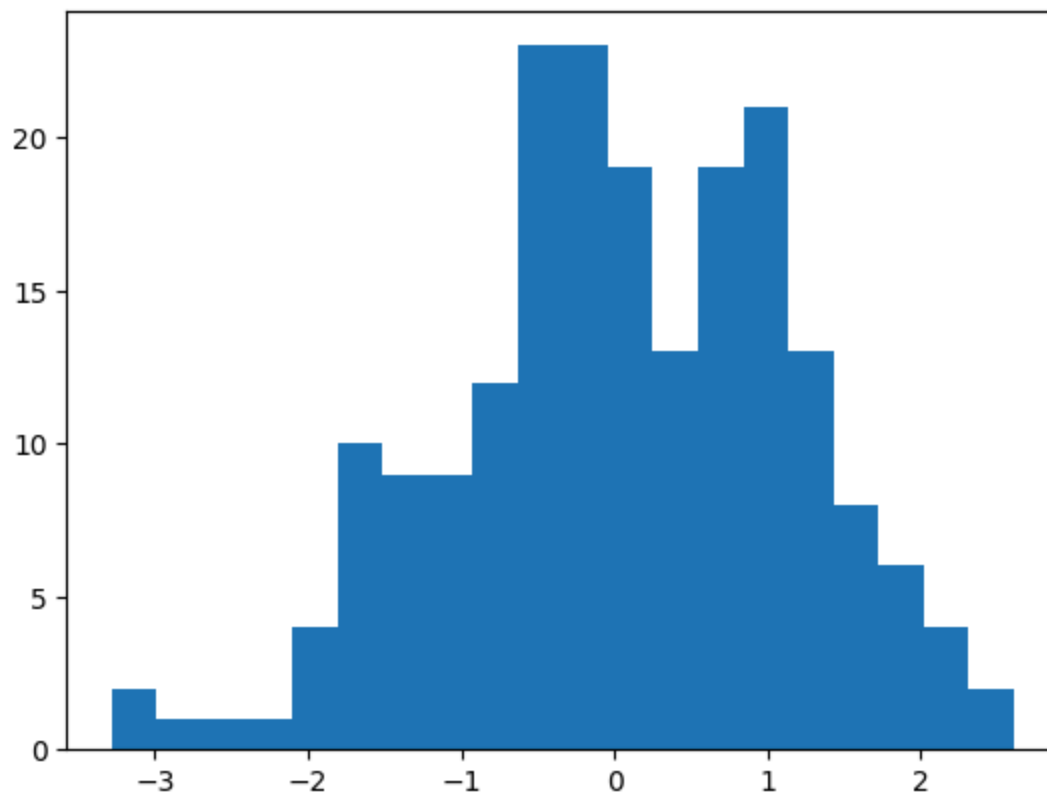


In [124... `np.max(noise)` *#maximum value produced by random functions*

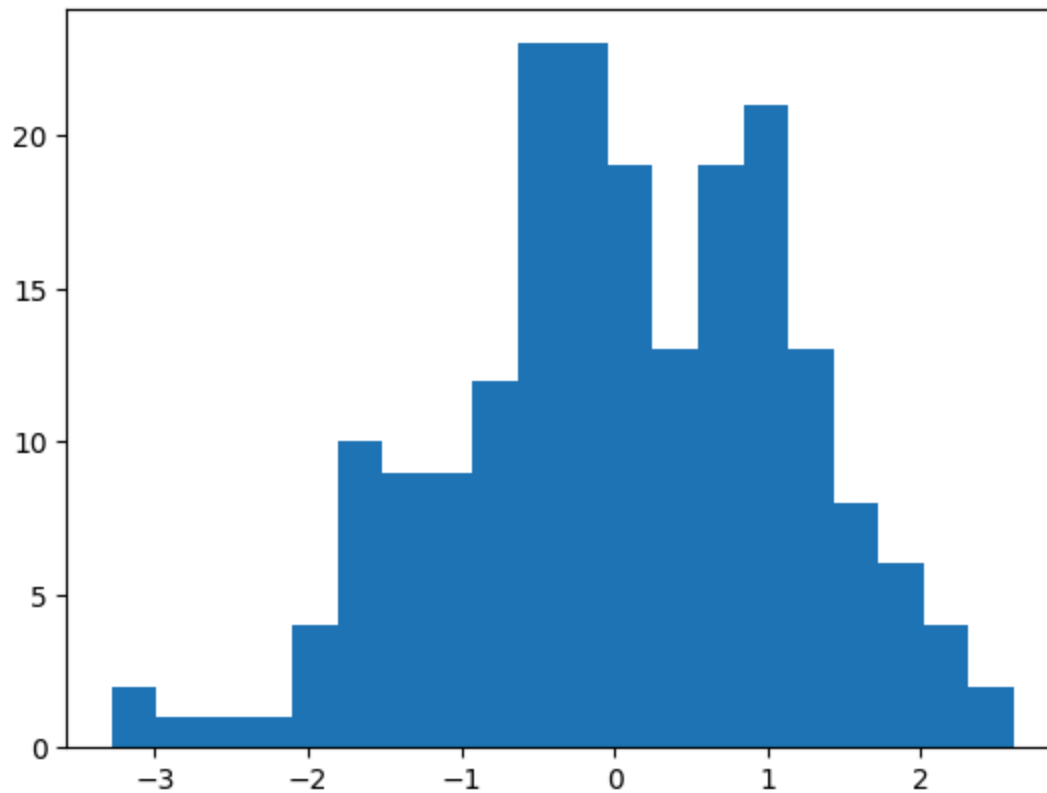
Out[124... `np.float64(2.6060952401529014)`

In [125... `hist(noise, 20)` *# histogram is used to display radom distributions;*
We plot a histogram with 20 intervals that start at the negative max and go to th
the first array contains the values of the verticals in the histogram,
and the second array are the coordinates of the intervals

Out[125... (array([2., 1., 1., 1., 4., 10., 9., 9., 12., 23., 23., 19., 13.,
19., 21., 13., 8., 6., 4., 2.]),
array([-3.28154914, -2.98716692, -2.69278471, -2.39840249, -2.10402027,
-1.80963805, -1.51525583, -1.22087361, -0.92649139, -0.63210917,
-0.33772695, -0.04334473, 0.25103749, 0.54541971, 0.83980193,
1.13418414, 1.42856636, 1.72294858, 2.0173308 , 2.31171302,
2.60609524])),
<BarContainer object of 20 artists>)



In [126... `_ = hist(noise, 20)`
if you don't want to print the values, use underscore;
convention in Python is to use _ for throwaway variables



In [127...

`print(_)`

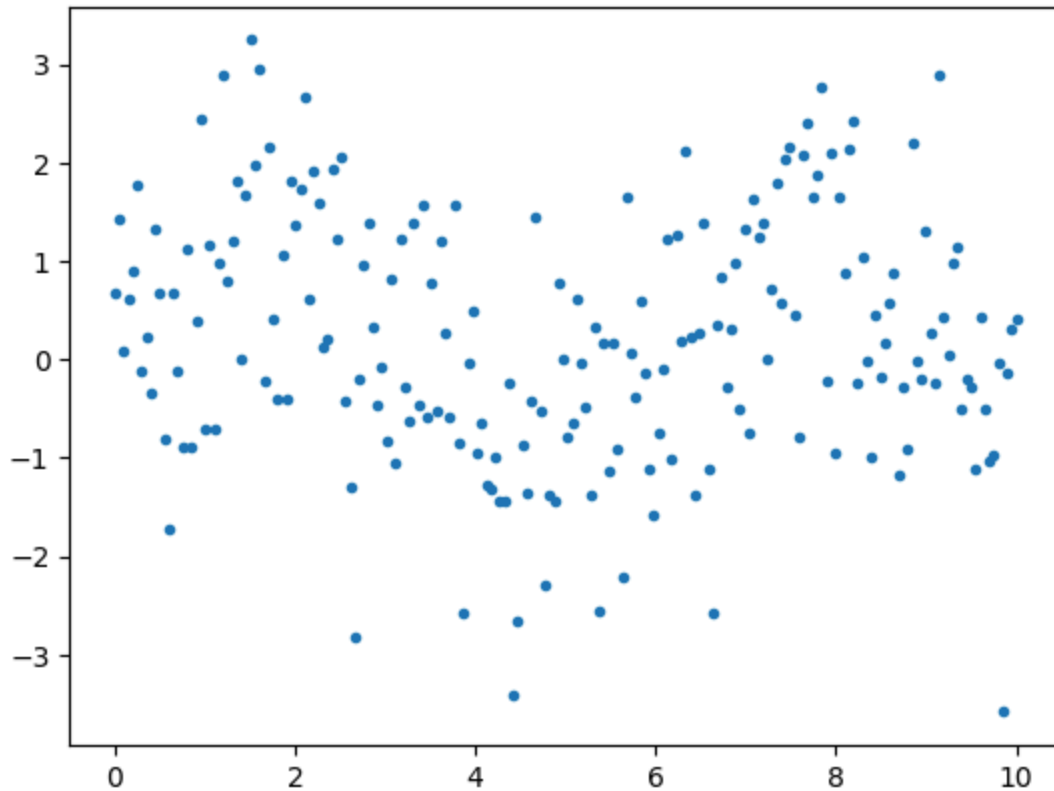
```
(array([ 2.,  1.,  1.,  1.,  4., 10.,  9.,  9., 12., 23., 23., 19., 13.,
        19., 21., 13.,  8.,  6.,  4.,  2.]), array([-3.28154914, -2.98716692, -2.6927
8471, -2.39840249, -2.10402027,
        -1.80963805, -1.51525583, -1.22087361, -0.92649139, -0.63210917,
        -0.33772695, -0.04334473,  0.25103749,  0.54541971,  0.83980193,
         1.13418414,  1.42856636,  1.72294858,  2.0173308 ,  2.31171302,
         2.60609524])), <BarContainer object of 20 artists>)
```

In [128...

```
obs = data + noise # a major component of our experimental data is the sin() function
# because our experiments are always stochastic,
# we added noise; in other words, our observed values (obs) are sin() plus noise
# we add arrays element by element
plot(x, obs, '.')
```

Out[128...

```
[<matplotlib.lines.Line2D at 0x271cf0bdd10>]
```

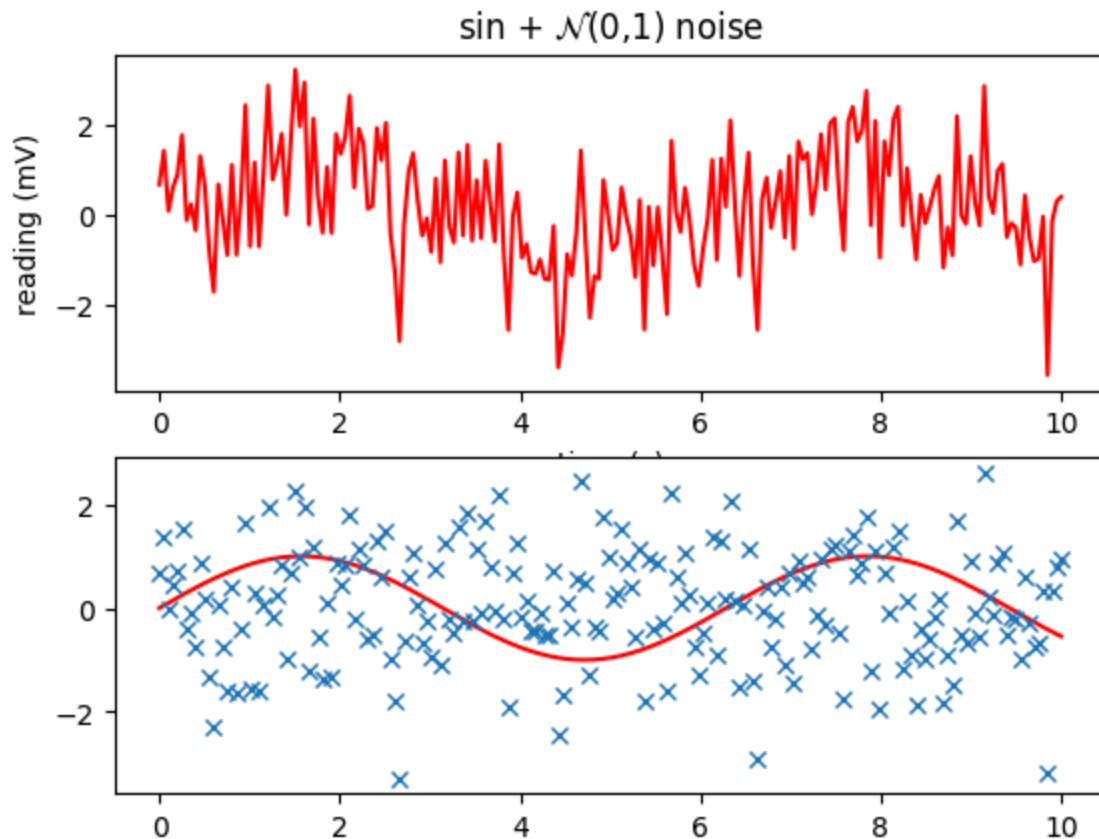


We can also show the `sin()` function superimposed with the noise

In [129...

```
obs = data + noise
figure()
subplot(211) # declares a subplot of two rows and one column and selects the first
#starting from top to bottom (left to right)
title('sin +  $\mathcal{N}(0,1)$  noise') #how you add titles to your displays
xlabel('time (s)') #how add labels to your graphs
ylabel('reading (mV)')
plot(x, obs, 'r')
subplot(212)
plot(x, data, 'r')
plot(x, noise, 'x')
show()
```

```
<>:5: SyntaxWarning: invalid escape sequence '\m'
<>:5: SyntaxWarning: invalid escape sequence '\m'
C:\Users\mleen\AppData\Local\Temp\ipykernel_34380\957691629.py:5: SyntaxWarning: inv
alid escape sequence '\m'
    title('sin +  $\mathcal{N}(0,1)$  noise') #how you add titles to your displays
```



Scipy

Python library used for scientific and technical computing. Contains modules for the following:

- Optimization & linear algebra
- Integration & interpolation
- Special functions (Bessel, ArcTan, Laguerre, etc...)
- Fast Fourier Transform (FFT)
- signal and image processing
- ODE solvers

Uses and expands on the capabilities of NumPy

The following example is a brief illustration of how convenient it is to use functions in Python and these packages, including Scipy.

In [130... `!pip install scipy`

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: scipy in c:\users\mleen\appdata\roaming\python\python313\site-packages (1.16.2)
Requirement already satisfied: numpy<2.6,>=1.25.2 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from scipy) (2.3.2)
```

```
[notice] A new release of pip is available: 25.2 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
In [131... import matplotlib.pyplot as plt
import numpy as np
import scipy as sp
import scipy.optimize
from numpy.random import rand
```

```
In [132... 'sp.optimize' in dir(sp) #checks if sp package contains optimized subfunction
```

```
Out[132... False
```

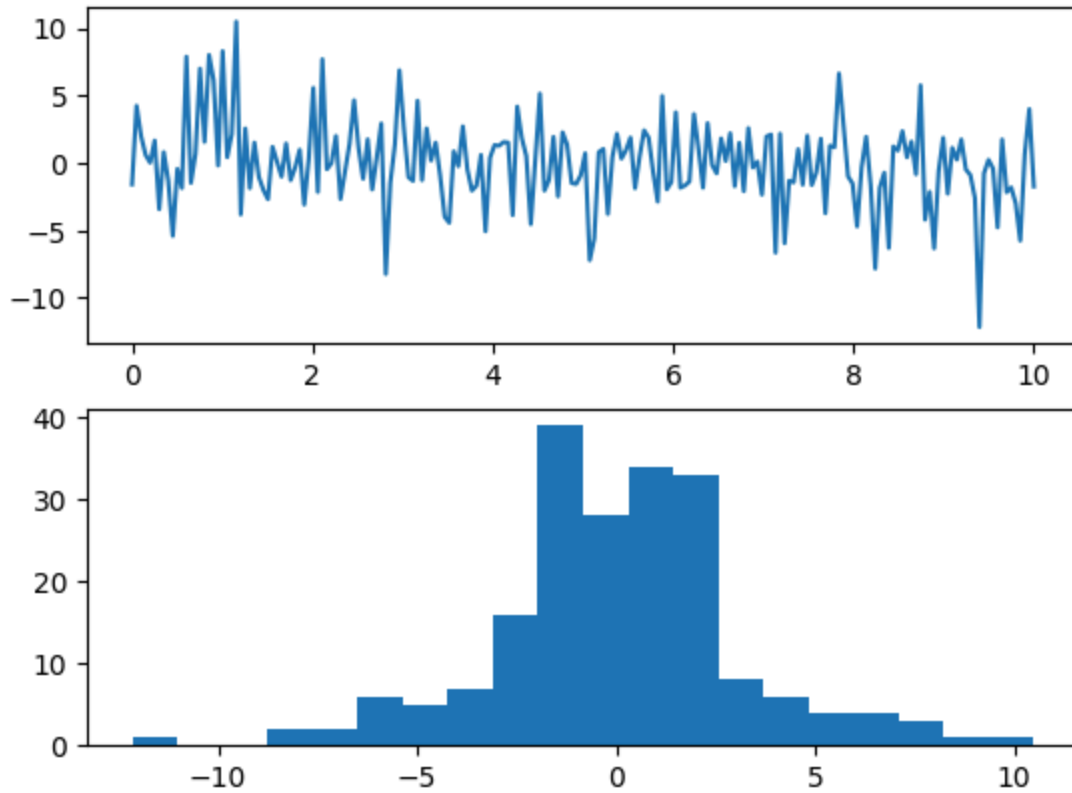
Again we want to show you how you can display various function. For example say we have a signal that can be described by the following function

$$signal = A \times \cos(2\pi x - 1.32) + 0.22x$$

```
In [133... points = 200
x      = np.linspace(0, 10, points) #defines points on the axis

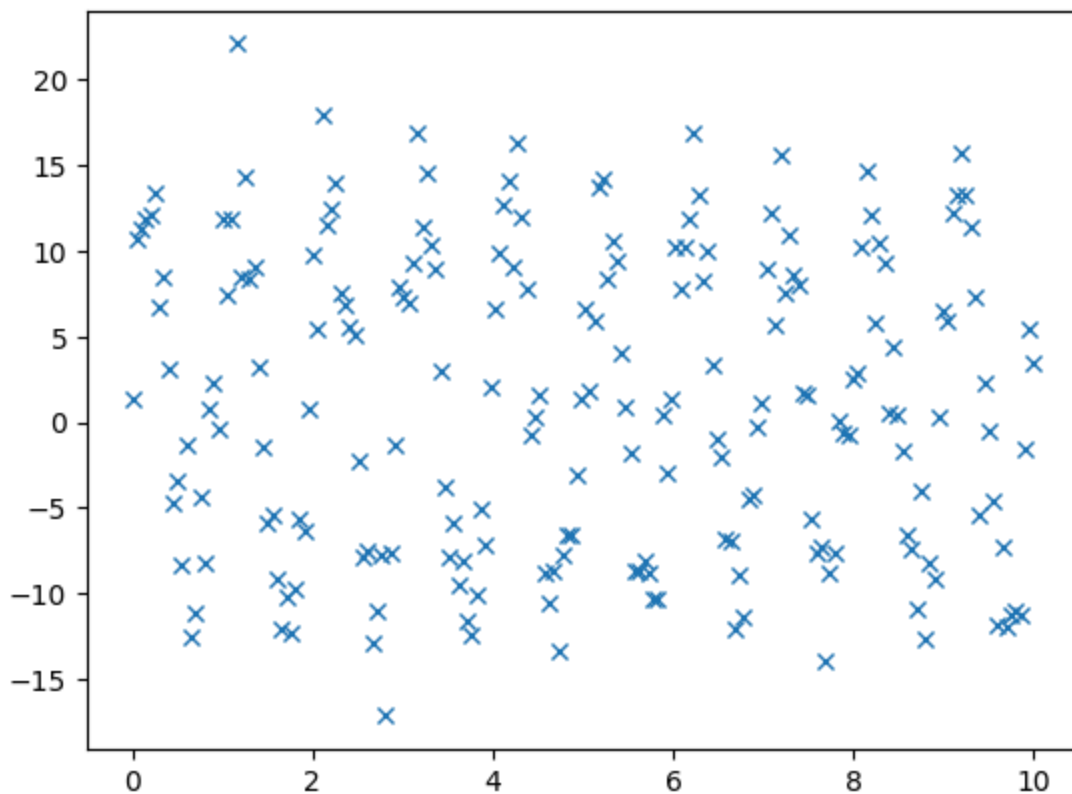
# func-> A*cos(2pi*x-B)+x*C+noise
signal = 12.0 * np.cos(2*np.pi*x-1.32) + 0.22 * x
# we don't actually see the linear function .22x in our plot because it grows slowl
noise  = 4 * ((0.5-rand(points)) * np.exp(2*rand(points)**2)) #generating random no
combined = signal + noise
```

```
In [134... figure()
subplot(211)
plot(x, noise)
subplot(212)
_ = hist(noise,20)
```



```
In [135... plot(x, combined, 'x')  
#plot(x, signal, 'o')
```

```
Out[135... [<matplotlib.lines.Line2D at 0x271e7360e10>]
```



We generated a `cos()` function with noise. Now, we need to find a function that is a best fit. Inside the optimization subpackage we have the least square fit function that identifies the best fitting periodic function for experimental data. We will call this function 'model' and it will have two variable: parameter 0 and parameter 1

$$model = p_0 \times \cos(2\pi x - p_1)$$

```
In [136... model = lambda p, x: p[0] * np.cos(2*np.pi*x-p[1])
# the function we are trying to extract
# lambda operator, creates anonymous throwaway function of argument p,x, output p[0]
# p is a vector of paramaters that we are looking for
err = lambda p, x, y: model(p, x) - y
# also a lambda function that finds the error in the model compared to our
# experimental data; you subtract y values that were generated above by the original
# function
```

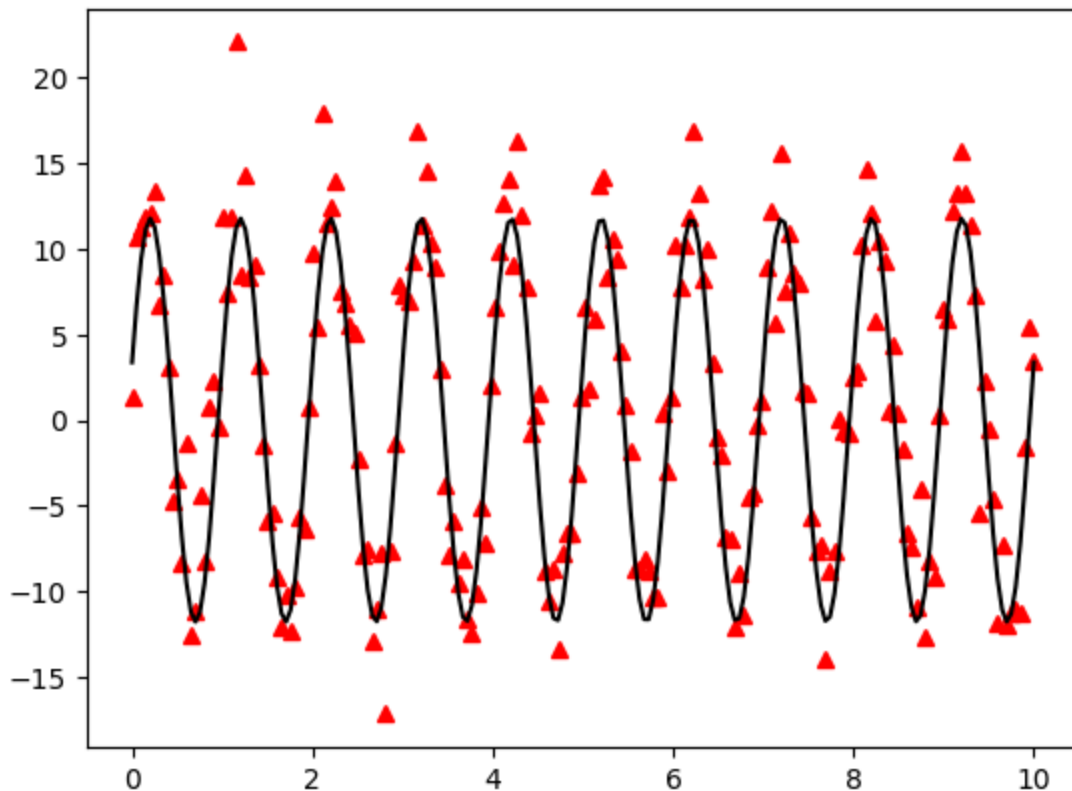
```
In [137... guess = [1., 0.8]
fitted, success = sp.optimize.leastsq(err, guess, args=(x, combined))
# guess = starting values for the function;
# the function calculates errors for value iteratively, trying to minimize error
# it minimizes the sum of squares of a set of equations
```

```
In [138... success
```

```
Out[138... 2
```

```
In [139... plt.plot(x, combined, 'r^')
plt.plot(x, model(fitted,x), 'k-')
# red is simulated experimental data;
#black lines is least square best fit (we used optimize function for that)
```

```
Out[139... [<matplotlib.lines.Line2D at 0x271e73b3ed0>]
```



Injest, analyze, and clean data with Pandas

Example 1: Car catalog

Simple analysis workflow of a well known sample data set of 400 cars from the 1970s.

In [140... `import pandas as pd`

```
pd.set_option("display.max_rows",10)
```

In [141... `from pandas.plotting import scatter_matrix`

```
import matplotlib.pyplot as plt
```

```
# Load the dataset of metrics for varios car models
```

```
file = r"C:\Users\mleen\michael_sandbox\metcs599\labs\lab1\Small_Car_Data.csv"
```

```
autompg = pd.read_csv(file)
```

```
no_rows = len(autompg.index)
```

```
no_columns = len(autompg.columns)
```

```
print("number of rows ",no_rows)
```

```
print("number of columns ", no_columns)
```

```
#attributes = ['Acceleration', 'Cylinders', 'Displacement', 'Horsepower', 'MPG']
```

```
#scatter_matrix(autompg[attributes], figsize=(12,8))
```

```
#plt.show();
```

number of rows 100
number of columns 11

In [142... autompg *#prints the data*

Out[142... **Unnamed: 0** Acceleration Cylinders Displacement Horsepower Manufacturer Model

0	1	12.0	8	307	130	chevrolet	chevrolet malibu
1	2	11.5	8	350	165	buick	buick skylark
2	3	11.0	8	318	150	plymouth	plymouth satellite
3	4	12.0	8	304	150	amc	amc rebel
4	5	10.5	8	302	140	ford	ford torino
...	...	...	...	...	...	...	...
95	96	15.6	4	140	86	ford	ford mustang
96	97	24.6	4	97	52	volkswagen	volkswagen pickup
97	98	11.6	4	135	84	dodge	dodge rampart
98	99	18.6	4	120	79	ford	ford ranger
99	100	19.4	4	119	82	chevrolet	chevrolet s-10

100 rows × 11 columns



In [143... type(autompg)

Out[143... pandas.core.frame.DataFrame

In [144... autompg.sort_values(by='MPG')

Out[144...

	Unnamed: 0	Acceleration	Cylinders	Displacement	Horsepower	Manufacturer	Model
34	35	18.5	8	304	193	ih	ih 120
31	32	14.0	8	360	215	ford	ford f2
32	33	15.0	8	307	200	chevrolet	chevro (
33	34	13.5	8	318	210	dodge	doc d2
61	62	13.2	8	318	150	plymouth	plymor vol premier
...	...	...	...	...	...	...	
87	88	16.2	4	91	67	datsun	datsun 3
89	90	17.0	6	262	85	oldsmobile	oldsmok cutl ci (dies
80	81	14.7	4	105	63	plymouth	plymor horiz mi
85	86	15.0	4	91	67	honda	honda ci
96	97	24.6	4	97	52	volkswagen	volkswag pick

100 rows × 11 columns



In [145...

```
autompg.sort_values(by='MPG',ascending=False) # ascending descending
```

Out[145...

	Unnamed: 0	Acceleration	Cylinders	Displacement	Horsepower	Manufacturer	Model
96	97	24.6	4	97	52	volkswagen	volkswagen pickup
85	86	15.0	4	91	67	honda	honda civic
89	90	17.0	6	262	85	oldsmobile	oldsmobile cutlass (diesel)
80	81	14.7	4	105	63	plymouth	plymouth horizon
87	88	16.2	4	91	67	datson	datson 510
...	...	...	...	...	...	...	...
68	69	14.0	8	318	150	dodge	dodge d150
33	34	13.5	8	318	210	dodge	dodge d250
31	32	14.0	8	360	215	ford	ford f250
32	33	15.0	8	307	200	chevrolet	chevrolet c1500
34	35	18.5	8	304	193	ih	ih 1200

100 rows × 11 columns



In [146...

```
attributes = ['Acceleration', 'Cylinders', 'Displacement', 'Horsepower', 'MPG']
# panda allows us to pick a few columns, not the whole set
automp[attributes]
```

Out[146...

	Acceleration	Cylinders	Displacement	Horsepower	MPG
0	12.0	8	307	130	18.0
1	11.5	8	350	165	15.0
2	11.0	8	318	150	18.0
3	12.0	8	304	150	16.0
4	10.5	8	302	140	17.0
...	...	...	...	...	...
95	15.6	4	140	86	27.0
96	24.6	4	97	52	44.0
97	11.6	4	135	84	32.0
98	18.6	4	120	79	28.0
99	19.4	4	119	82	31.0

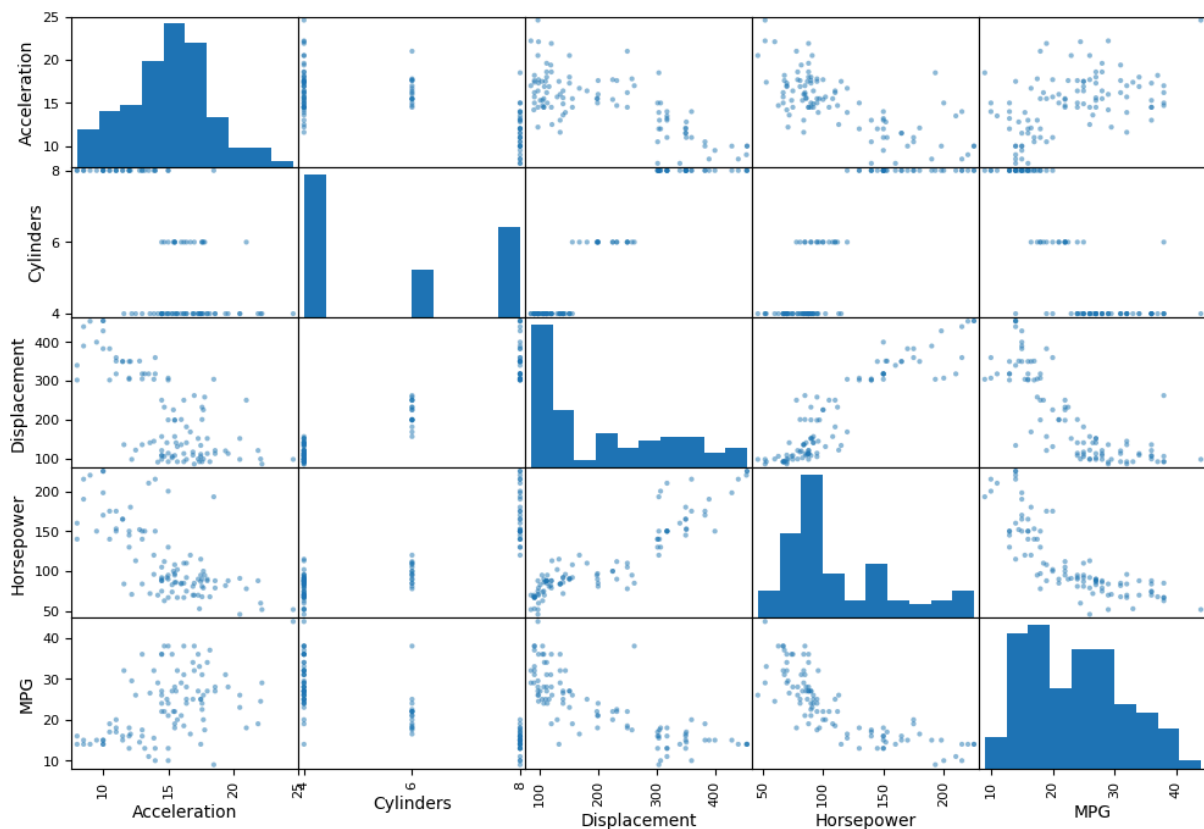
100 rows × 5 columns

In [147...

```

attributes = ['Acceleration', 'Cylinders', 'Displacement', 'Horsepower', 'MPG']
scatter_matrix(autmpg[attributes], figsize=(12,8))
plt.show()

```



On the diagonal of the scatter matrix we see (statistical) distributions of values in respective column. The off diagonal plots present values in the row attribute as a function of values in the column attribute. Those off diagonal plots coupled with covariance matrices for each pair of variables (attributes) tell us about mutual correlation between two variable (attributes). For example, Displacement and Horsepower are positively correlated. Displacement and MPG are negatively correlated. A broad cloud means little correlation between two variables.

In [148...

autompg

Out[148...

	Unnamed: 0	Acceleration	Cylinders	Displacement	Horsepower	Manufacturer	Model
0	1	12.0	8	307	130	chevrolet	chevrolet malibu
1	2	11.5	8	350	165	buick	buick skylark 3
2	3	11.0	8	318	150	plymouth	plymouth satellite
3	4	12.0	8	304	150	amc	amc rebel
4	5	10.5	8	302	140	ford	ford torino
...	...	...	...	...	...	...	...
95	96	15.6	4	140	86	ford	ford mustang
96	97	24.6	4	97	52	volkswagen	volkswagen pickup
97	98	11.6	4	135	84	dodge	dodge rampart
98	99	18.6	4	120	79	ford	ford ranger
99	100	19.4	4	119	82	chevrolet	chevrolet s-

100 rows × 11 columns



In [149...

sorted(autompg.Manufacturer.unique())

```
Out[149... ['amc',  
            'audi',  
            'bmw',  
            'buick',  
            'cadillac',  
            'chevrolet',  
            'chrysler',  
            'citroen',  
            'datsum',  
            'dodge',  
            'fiat',  
            'ford',  
            'honda',  
            'ih',  
            'mazda',  
            'mercedes-benz',  
            'mercury',  
            'nissan',  
            'oldsmobile',  
            'opel',  
            'peugeot',  
            'plymouth',  
            'pontiac',  
            'renault',  
            'saab',  
            'toyota',  
            'volkswagen',  
            'volvo']
```

```
In [150... sorted(autompg.Manufacturer.value_counts())
```



```
Out[150...] [1,
             1,
             1,
             1,
             1,
             1,
             1,
             1,
             1,
             1,
             1,
             1,
             1,
             1,
             1,
             1,
             1,
             1,
             1,
             1,
             2,
             2,
             2,
             3,
             3,
             4,
             4,
             5,
             5,
             8,
             9,
             9,
             15,
             16]
```

Several makes have spelling errors or inconsistencies. Pandas can help us fix those in place:

```
In [151...] autompg.loc[autompg.Manufacturer == 'chevroelt', 'Manufacturer'] = 'chevrolet'
autompg.loc[autompg.Manufacturer == 'chevy', 'Manufacturer'] = 'chevrolet'
autompg.loc[autompg.Manufacturer == 'maxda', 'Manufacturer'] = 'mazda'
autompg.loc[autompg.Manufacturer == 'mercedes', 'Manufacturer'] = 'mercedes-benz'
autompg.loc[autompg.Manufacturer == 'toyouta', 'Manufacturer'] = 'toyota'
autompg.loc[autompg.Manufacturer == 'vokswagen', 'Manufacturer'] = 'volkswagen'
autompg.loc[autompg.Manufacturer == 'vw', 'Manufacturer'] = 'volkswagen'
```

```
In [152...] sorted(autompg.Manufacturer.unique())
```

```
Out[152... ['amc',  
            'audi',  
            'bmw',  
            'buick',  
            'cadillac',  
            'chevrolet',  
            'chrysler',  
            'citroen',  
            'datsun',  
            'dodge',  
            'fiat',  
            'ford',  
            'honda',  
            'ih',  
            'mazda',  
            'mercedes-benz',  
            'mercury',  
            'nissan',  
            'oldsmobile',  
            'opel',  
            'peugeot',  
            'plymouth',  
            'pontiac',  
            'renault',  
            'saab',  
            'toyota',  
            'volkswagen',  
            'volvo']
```

```
In [153... with pd.option_context('display.max_rows', 999):  
            print(autompg.groupby('Manufacturer').size())
```

```
Manufacturer
amc          9
audi         1
bmw          1
buick        3
cadillac     1
chevrolet    15
chrysler     1
citroen      1
datsun       3
dodge        9
fiat         1
ford         16
honda        4
ih           1
mazda        2
mercedes-benz 1
mercury      1
nissan        1
oldsmobile   1
opel         1
peugeot      2
plymouth     8
pontiac      4
renault      1
saab         1
toyota       5
volkswagen   5
volvo        1
dtype: int64
```

Example 2: Heat Map Representation of Correlations Between Variables

Pearson Correlations

Seaborn is another popular Graphics Library like Matplotlib. We will demonstrate its heat-map functions

```
In [154... !pip install seaborn
```

Defaulting to user installation because normal site-packages is not writeable
 Requirement already satisfied: seaborn in c:\users\mleen\appdata\roaming\python\python313\site-packages (0.13.2)
 Requirement already satisfied: numpy!=1.24.0,>=1.20 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from seaborn) (2.3.2)
 Requirement already satisfied: pandas>=1.2 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from seaborn) (2.3.2)
 Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from seaborn) (3.10.6)
 Requirement already satisfied: contourpy>=1.0.1 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)
 Requirement already satisfied: cyclor>=0.10 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
 Requirement already satisfied: fonttools>=4.22.0 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.59.2)
 Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.9)
 Requirement already satisfied: packaging>=20.0 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (25.0)
 Requirement already satisfied: pillow>=8 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (11.3.0)
 Requirement already satisfied: pyparsing>=2.3.1 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.2.3)
 Requirement already satisfied: python-dateutil>=2.7 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
 Requirement already satisfied: pytz>=2020.1 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from pandas>=1.2->seaborn) (2025.2)
 Requirement already satisfied: tzdata>=2022.7 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from pandas>=1.2->seaborn) (2025.2)
 Requirement already satisfied: six>=1.5 in c:\users\mleen\appdata\roaming\python\python313\site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)

[notice] A new release of pip is available: 25.2 -> 25.3

[notice] To update, run: python.exe -m pip install --upgrade pip

In [155...

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
sns.set(context="paper", font="monospace")
#sns.palplot(sns.color_palette("hls",4))

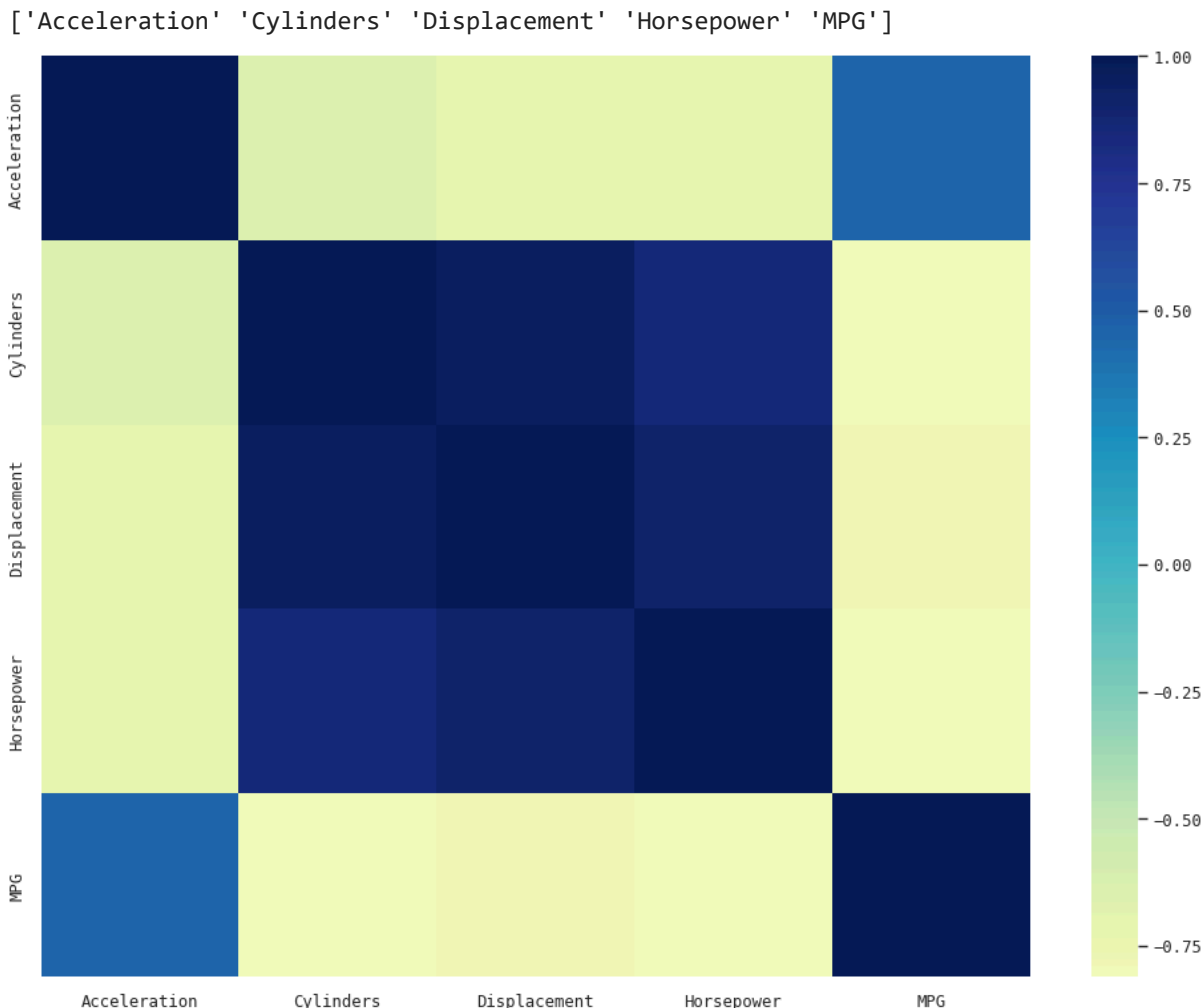
small_auto = autmpg[attributes]
#no_rows = len(small_auto.index)
#no_columns = len(small_auto.columns)
#print("number of rows ",no_rows)
#print("number of columns ", no_columns)
print( small_auto.columns.values)

corrmat = small_auto.corr(method='pearson')

# Set up the matplotlib figure
f, ax = plt.subplots(figsize=(12, 9))

# Draw the heatmap using seaborn
```

```
sns.heatmap(corrmat,center=0, cmap="YlGnBu")
plt.show()
```



Positively correlated attributes intersect at dark blue squares. Negatively correlated attributes intersect and off-white squares.

The above Heat-map actually represents well known statistical measures known as Pearson Correlation Coefficients. The text below is from Wikipedia.org.

Pearson's correlation coefficient when applied to a population is commonly represented by the Greek letter ρ (rho) and may be referred to as the population correlation coefficient or the population Pearson correlation coefficient. The formula for ρ is:

$$\rho_{X,Y} = \frac{\operatorname{cov}(X,Y)}{\sigma_X \sigma_Y}$$

where: cov is the covariance, σ_X is the standard deviation of X , σ_Y is the standard deviation of Y . The formula for ρ can be expressed in terms of mean and expectation. Since $\operatorname{cov}(X,Y) = E[(X - \mu_X)(Y - \mu_Y)]$, the formula for ρ can also be written as:

$$\rho_{X,Y} = \frac{E[(X - \mu_X)(Y - \mu_Y)]}{\sigma_X \sigma_Y}$$

$$\rho_{X,Y} = \frac{\operatorname{E}[(X - \mu_X)(Y - \mu_Y)]}{\sigma_X \sigma_Y}$$
 where: σ_X and σ_Y are defined as above μ_X is the mean of X E is the expectation.

In []: